

第1章: 開発環境のセットアップ

この章では、書籍管理Webアプリケーションを開発するために必要なツールをすべてインストールし、正しく動作することを確認します。プログラミングが初めての方でも迷わないよう、手順を一つひとつ丁寧に解説します。

この章で行うこと

「開発環境のセットアップ」とは、プログラムを書いて動かすために必要なソフトウェアをパソコンにインストールする作業です。料理に例えると、実際に料理を始める前に「包丁・まな板・鍋」を揃えるようなものです。

具体的には、以下の4つのツールをインストールします。

ツール	料理で例えると	なぜ必要か
Node.js	ガスコンロ（調理の火力源）	プログラムを動かすエンジン。これがないと何も始まりません
VS Code	作業台（まな板と包丁のセット）	プログラムを書くためのエディタ。メモ帳でも書けますが、VS Codeは間違いを指摘してくれたり、コードに色を付けてくれたりする優秀な道具です
Git	レシピノート	コードの変更履歴を記録するツール。「さっきの状態に戻したい」が簡単にできます
ターミナル	キッチン全体	すべてのツールを操作するための入口。文字を打ってパソコンに指示を出します

用語メモ: - **ターミナル**（terminal：端末）／**コマンドライン**（command line：コマンド行）／**シェル**（shell：殻）／**CLI**（Command Line Interface：コマンドラインインターフェース）はほぼ同じ意味で使われます。すべて「文字を打ってパソコンに命令する画面」を指す言葉です。本書ではまとめて「ターミナル」と呼びます。 - **GUI**（Graphical User Interface：グラフィカルユーザーインターフェース）はマウスでアイコンをクリックして操作する、いつものWindowsやMacの画面のことです。

所要時間の目安： 30～60分（ダウンロードの待ち時間を含む）

目次

1. Node.js のインストール
2. パッケージマネージャーの理解
3. コードエディタ (VS Code)
4. Git の基礎

1. Node.js のインストール

1.1 Node.js とは何か

Node.js（ノードジェイエス）は、JavaScript（ジャバスクリプト：もともとはWebブラウザ上で動くプログラミング言語）をブラウザの外（サーバーやあなたのPC）で実行できるようにする**ランタイム環境**（Runtime Environment：プログラムを実行するための土台ソフトウェア）です。

もともとJavaScriptはブラウザの中だけで動く言語でしたが、2009年にNode.jsが登場したことで、サーバーサイド（server side：サーバー側、ユーザーから見えない裏側の処理）やコマンドラインツール（CLI：キーボードで文字を打って操作するプログラム）の開発にも使えるようになりました。

なぜNode.jsが重要なのか： 現代のWeb開発では、プログラムを書いた後に「TypeScriptをJavaScriptに変換する」「複数のファイルを1つにまとめる」「開発用サーバーを起動する」といった様々な処理が必要です。これらはすべてNode.js上で動きます。つまり、Node.jsは**開発の土台**となるソフトウェアです。

なぜ Node.js が必要なのか

書籍管理アプリの開発では、以下の場面で Node.js を使います。

用途	説明
フロントエンド開発サーバー	React などのフレームワークをローカル（自分のPC）で動かす
パッケージ管理	npm を使ってライブラリ（他人が作った便利なコード）をインストールする
ビルドツール	TypeScript のコンパイル（変換）、バンドル（複数ファイルの結合）など
バックエンド API サーバー	Express.js など REST API（データをやり取りする窓口）を構築する
データベース操作	Prisma などの ORM（データベースとプログラムをつなぐ翻訳ツール）を使ってデータベースとやり取りする

開発ワークフローにおける Node.js の位置づけ



1.2 インストール方法

Node.js のインストールには2つの方法があります。

1. 公式サイトから直接インストール — 最もシンプルな方法
2. nvm (Node Version Manager) を使う — 推奨される方法

💡 **ヒント:** 本チュートリアルでは **nvm** を使ったインストールを強く推奨します。理由は、プロジェクトによって必要な Node.js のバージョンが異なることがあり、nvm を使えばバージョンの切り替えが簡単にできるからです。

nvm (エヌブイエム: **N**ode **V**ersion **M**anager) とは: Node.js の「複数バージョン管理ツール」のこと。Aプロジェクトでは Node.js 18、Bプロジェクトでは Node.js 20、というように1台のPCで異なるバージョンを使い分けられます。

方法 A: nvm を使ったインストール（推奨）

Windows の場合

Windows では `nvm-windows` を使います（Linux/Mac 用の nvm とは別のツールです）。

手順 1: nvm-windows のダウンロード

1. `nvm-windows` リリースページ にアクセスします
2. 最新版の `nvm-setup.exe` をダウンロードします（`.exe` はWindowsの実行ファイル形式）
3. ダウンロードしたインストーラーを実行します（ダブルクリックで起動）

手順 2: インストーラーの実行

1. 「I accept the agreement」を選択して「Next」
2. インストール先はデフォルトのまま「Next」
（通常: `C:\Users\<ユーザー名>\AppData\Roaming\nvm`）
3. Node.js のシンボリックリンク先もデフォルトのまま「Next」
（通常: `C:\Program Files\nodejs`）
4. 「Install」をクリック
5. 「Finish」をクリック

用語メモ: - シンボリックリンク（symbolic link）：ファイルやフォルダの「ショートカット」のようなもの。nvm が現在使うNode.jsを切り替えるとき、このショートカット先を入れ替える仕組みになっています。 - `C:\Users\<ユーザー名>` : Windowsで自分の個人用フォルダ（ホームフォルダ）。`<ユーザー名>` の部分はあなたのWindowsアカウント名に置き換わります（例: `C:\Users\yuya`）。環境変数 `%USERPROFILE%` でも同じ場所を指します。 - `AppData\Roaming` : アプリの設定データが入る隠しフォルダ。通常エクスプローラーでは見えません。

手順 3: Node.js のインストール

新しい PowerShell またはコマンドプロンプトを **管理者として** 開き、以下を実行します。

管理者として実行とは: Windowsでは「普通のユーザー権限」と「管理者権限」の2段階があります。システムフォルダ（`C:\Program Files` など）を書き換える操作には管理者権限が必要です。PowerShellのアイコンを**右クリック** → 「**管理者として実行**」を選ぶと、青いタイトルバーの管理者用ウィンドウが開きます。

```
# nvm が正しくインストールされたか確認。バージョン番号が表示されればOK
nvm version

# インストール可能な Node.js のバージョン一覧をネット経由で取得して表示する
# available = 「利用可能な」という意味
nvm list available

# LTS (Long Term Support : 長期サポート版) の最新版をダウンロードしてインストール
# LTS版は数年間バグ修正が続く安定版。仕事ではLTSを使うのが普通
nvm install lts

# インストール済みのうち LTS 版を「現在使うバージョン」として有効化
# nvm は複数バージョンを共存できるが、同時に使うのは1つだけなので切替が必要
nvm use lts

# 確認: Node.js本体のバージョンを表示。-v は --version の短縮形
node -v

# 確認: npm (次の節で説明するパッケージ管理ツール) のバージョンを表示
npm -v
```

⚠ **注意:** Windows では `nvm install` と `nvm use` コマンドの実行に **管理者権限** が必要です。

PowerShell を右クリックして「管理者として実行」を選んでください。

インストール失敗時の典型原因: - 管理者として実行していない → 上記の通り右クリックから起動し直す - ウイルス対策ソフトがブロック → 一時的にオフにして再試行 - 社内ネットワークやプロキシ → 会社支給PCの場合、IT担当者にご相談

Mac の場合

手順 1: nvm のインストール

ターミナル (macOS標準アプリの **Terminal.app** : アプリケーション → ユーティリティ にあります) を開いて以下を実行します。

```
# nvm の公式インストールスクリプトを curl でダウンロードしてシェルで実行する
# curl: ネット上のファイルを取ってくるコマンド
# -o- : ダウンロードしたデータをファイルに保存せず、標準出力 (次のコマンド) に流す
# | : パイプ。左コマンドの出力を右コマンドの入力につなぐ
# bash : シェル本体。受け取ったスクリプトを解釈して実行する
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
```

手順 2: シェルの再起動

インストール後、ターミナルを一度閉じて再度開きます。または以下を実行します。

シェル設定ファイルとは: ターミナルを開いたときに自動で読み込まれる設定ファイル。`~/.zshrc` や `~/.bashrc` がそれ。`~` はあなたのホームフォルダを指す記号。インストーラーがこのファイルに `nvm` の読み込み行を追記しているので、再読み込みが必要。

```
# zsh (macOS Catalina以降のデフォルトシェル) を使っている場合
# source: 指定ファイルを「いま開いているシェル」で再読み込みするコマンド
# ~/.zshrc: ホームフォルダ直下にある zsh の設定ファイル
source ~/.zshrc

# bash (古いmacOSや一部Linuxのデフォルト) を使っている場合
# ~/.bashrc: ホームフォルダ直下にある bash の設定ファイル
source ~/.bashrc
```

手順 3: Node.js のインストール

```
# nvm 自体が使えるか確認。バージョン番号が表示されればインストール成功
nvm --version

# LTS (長期サポート) 版のNode.jsをインストール
# --lts はロングフラグ (長い名前のオプション)。意味は Windows の `lts` と同じ
nvm install --lts

# 確認: Node.js のバージョン表示
node -v

# 確認: npm のバージョン表示
npm -v
```

💡 ヒント: Mac で Homebrew (Macの追加ソフト管理ツール) を使っている場合は `brew install nvm` でもインストールできますが、公式のインストールスクリプトを使う方が確実です。

```
# システムのパッケージ情報を最新化する
# sudo : 一時的に管理者権限で実行 (Super User DO の略)
# apt : Debian/Ubuntu系の標準パッケージ管理コマンド
# update: パッケージ一覧を最新化 (実際のインストールはまだしない)
sudo apt update

# curl (ダウンロードコマンド) をインストール
# install: 指定パッケージをダウンロード&インストール
# -y : 確認プロンプトすべてに「yes」と自動回答するフラグ
sudo apt install curl -y

# nvm の公式インストールスクリプトをダウンロード&実行
# curl -o- でデータを取得し、パイプ | で bash に流して実行する
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash

# .bashrc (bash用シェル設定ファイル) を現在のシェルで再読み込み
# これで nvm コマンドが使えるようになる
source ~/.bashrc

# nvm がインストールできたか確認
nvm --version

# Node.js の LTS 版をインストール
nvm install --lts

# nvm alias: 別名 (エイリアス) を設定するサブコマンド
# default node: 「ターミナルを新しく開いたとき、最新版のnodeを自動で使う」設定
# これがないと、ターミナルを開くたびに `nvm use --lts` を打つ羽目になる
nvm alias default node

# 確認
node -v
npm -v
```

方法 B: 公式サイトから直接インストール

nvm を使わない場合は、公式サイトから直接インストールできます。

1. [Node.js 公式サイト](#) にアクセスします
2. LTS (推奨版) をクリックしてダウンロードします
3. ダウンロードしたインストーラーを実行します

```
Windows: .msi ファイル (Microsoft Installerの略) を実行 → 画面の指示に従う
Mac:     .pkg ファイル (macOSのインストーラー形式) を実行 → 画面の指示に従う
Linux:   公式リポジトリを追加してインストール (下記参照)
```

Linux (Ubuntu/Debian) で公式リポジトリを使う場合:

```
# NodeSource (Node.js公式の配布元) リポジトリ追加スクリプトを実行する
# curl -fsSL は4つのフラグを連結:
#   -f : サーバーエラー時に静かに失敗 (Fail silently)
#   -s : 進捗バーを表示しない (Silent)
#   -S : -s 中でもエラーは表示する (Show errors)
#   -L : リダイレクト (転送) に追従する (Location)
# setup_20.x : Node.js のメジャーバージョン20系を指定
# | : 左で取得したシェルスクリプトを右のコマンドに渡す
# sudo -E bash - : 環境変数を保持(-E)したまま管理者権限のbashで実行。- は「標準入力を読む」の意味
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# 上のスクリプトでリポジトリ追加が済んだので、apt で nodejs をインストール
# -y : すべての確認プロンプトに自動でyes
sudo apt install -y nodejs

# 確認: バージョン表示
node -v
npm -v
```

1.3 バージョン確認

インストールが完了したら、ターミナル (Windows は PowerShell、Mac/Linux はターミナル) で以下を実行して確認します。

```
# Node.js 本体のバージョンを表示する
# -v は --version の短い書き方。多くのCLIツールで共通の慣習
node -v
# 出力例: v20.11.0
# 先頭のvは version の v。続く数字は「メジャー.マイナー.パッチ」の3階層

# npm (Node.jsに同梱されているパッケージ管理ツール) のバージョンを表示
npm -v
# 出力例: 10.2.4

# Node.js が実際にコードを実行できるか1行スクリプトでテスト
# -e "... " は eval の e。引用符内のJavaScriptコードをその場で実行するフラグ
# console.log() はカッコ内の文字を画面に出力する関数
node -e "console.log('Hello, Node.js!')"
# 出力: Hello, Node.js!
```

 **ヒント:** 本チュートリアルでは **Node.js 18.x 以上** を推奨します。 **node -v** で表示されるバージョンが **v18.0.0** 以上であることを確認してください。

1.4 トラブルシューティング

「node」コマンドが見つからない

'node' は、内部コマンドまたは外部コマンド、操作可能なプログラムまたはバッチ ファイルとして認識されていません。

原因: Node.js への実行ファイルの場所（パス）が 環境変数 PATH に登録されていません。

環境変数 PATH とは: OS（WindowsやMac）が「コマンドが入力されたとき、どのフォルダから実行ファイルを探すか」を記したリスト。node と打ったときに C:\Program Files\nodejs\node.exe が見つけれないと、上記エラーになります。

解決方法（Windows）：

```
# 1. 現在の PATH をセミコロン区切りで分割して見やすく表示
# $env:PATH : PowerShellで環境変数PATHを参照する書き方（$env: は環境変数のプレフィックス）
# -split ";" : 文字列をセミコロンで分割して配列にする演算子
$env:PATH -split ";"

# 2. Node.js のインストール先を確認
# 通常は以下のいずれか
# - C:\Program Files\nodejs\
# - C:\Users\<ユーザー名>\AppData\Roaming\nvm\<バージョン>\

# 3. 環境変数を設定（システムのプロパティ → 環境変数 → PATH に追加）
# または PowerShell で一時的に追加（再起動で消える）：
# += は「右辺を足して代入」する演算子。既存の PATH に新パスを末尾追加
$env:PATH += ";C:\Program Files\nodejs\"

# 4. ターミナルを再起動して再度確認
node -v
```

解決方法（Mac/Linux）：

```

# 1. node コマンドの実体がどこにあるか調べる
# which: 指定したコマンドの実行ファイルのフルパスを表示
which node
# または
# ls: フォルダ内のファイル一覧表示。ここではファイル単体を指定し、存在チェックに使う
ls /usr/local/bin/node

# 2. nvm を使っている場合、シェル設定ファイルに以下が追加されているか確認
# cat: ファイル内容を画面に表示
# | : パイプ。左の出力を右の入力に渡す
# grep "NVM": 「NVM」という文字列を含む行だけ抽出するコマンド
cat ~/.bashrc | grep NVM
# または zsh の場合
cat ~/.zshrc | grep NVM

# 以下のような行があるはず:
# export NVM_DIR="$HOME/.nvm"
# [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

# 3. なければ手動で追加
# echo "...": 文字列を出力するコマンド
# >> : 追記リダイレクト。出力をファイル末尾に追加 (> は上書き、>> は追記)
# シングルクォート '...' で囲むと、$変数 を展開せず文字通り書き込める
echo 'export NVM_DIR="$HOME/.nvm"' >> ~/.zshrc
echo '[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"' >> ~/.zshrc
# 設定ファイルを再読み込みして、その場で反映
source ~/.zshrc

```

Node.js のバージョンが古い

```

# 現在のバージョン確認
node -v
# 出力例: v14.17.0 ← 古すぎる (v18未満は本書で非対応)

# nvm を使っている場合: LTS版を取得 → 切り替え
nvm install --lts      # 最新LTSをダウンロード&インストール
nvm use --lts          # 「いま使うNode.js」をLTSに切り替え

# nvm を使っていない場合
# → 公式サイトから最新 LTS 版をダウンロードして再インストール

```

nvm コマンドが見つからない (Mac/Linux)

```
# シェル設定ファイルに NVM の設定があるか確認
# zsh の場合
# -A2: 「After 2」 マッチ行の後ろ2行も一緒に表示するフラグ。設定行は3行構成なので必要
cat ~/.zshrc | grep -A2 NVM

# 設定がなければ追加
# cat >> file << 'EOF' ... EOF はヒアドキュメント構文。
# EOF までの複数行をまとめてファイル末尾に追記する。
# 'EOF' (シングルクォート付き) にすると、$変数 を展開せず文字通り書き込める
cat >> ~/.zshrc << 'EOF'
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
EOF

# 設定を反映: 現在のシェルで .zshrc を再読み込み
source ~/.zshrc
```

npm install 時に EACCES エラー (Mac/Linux)

```
npm ERR! Error: EACCES: permission denied
```

EACCESとは: Error: ACCESS の略で「アクセス権限がない」というUNIX系OSの定型エラー。書き込み権限のないフォルダに書き込もうとしたときにします。

```
# nvm を使っていればこのエラーは通常発生しません
# グローバルインストール先を、書き込み権限がある場所に変更する手順:

# 1. ホームフォルダに専用フォルダを作る
# mkdir: フォルダ作成。~ はホームフォルダ
mkdir ~/.npm-global

# 2. npm の設定でグローバルインストール先 (prefix) を上で作ったフォルダに変更
# npm config set <キー> <値> : 設定を書き換えるコマンド
npm config set prefix '~/.npm-global'

# 3. PATH に上記フォルダの bin (実行ファイル置き場) を追加するよう .bashrc に追記
# $PATH: 既存のPATHを保持しつつ前方に新しいパスを足す書き方
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc

# 4. .bashrc を再読み込みして反映
source ~/.bashrc
```

2. パッケージマネージャーの理解

2.1 パッケージマネージャーとは

パッケージマネージャーは、プロジェクトで使うライブラリ（パッケージ：他人が作って公開した便利なコード部品）のインストール、更新、削除を管理するツールです。たとえば、「React」や「Express」などのライブラリをコマンド一つでインストールできます。

用語メモ: - パッケージ（package）／ライブラリ（library）：他人が作って公開してくれた、再利用可能なコード部品の集合。 - 依存関係（dependency：ディペンデンス）：「Aを動かすためにBが必要」というつながりのこと。Aは「Bに依存している」と言う。



レジストリ（registry：登録簿）とは: パッケージが世界中から集まる「公式サイト＝中央倉庫」。npm レジストリ（npmjs.com）には100万以上のパッケージが登録されており、`npm install` するとここからダウンロードされます。

2.2 npm vs yarn vs pnpm 比較

特徴	npm	yarn	pnpm
開発元	npm, Inc. (GitHub/Microsoft)	Meta (旧Facebook)	コミュニティ
Node.js 同梱	はい (デフォルト)	いいえ (別途インストール)	いいえ (別途インストール)
インストール速度	普通	速い	最も速い
ディスク使用量	多い	多い	少ない (ハードリンク)
ロックファイル	<code>package-lock.json</code>	<code>yarn.lock</code>	<code>pnpm-lock.yaml</code>
ワークスペース対応	v7 以降対応	対応	対応
学習コスト	低い	低い	やや高い
初心者おすすめ度	★★★★★	★★★★☆	★★★☆☆

💡 ヒント: 本チュートリアルでは **npm** を使用します。Node.js をインストールすると自動的に npm もインストールされるため、追加のセットアップが不要です。

2.3 package.json の理解

`package.json` はプロジェクトの「設計図」のようなファイルです。プロジェクトの名前、バージョン、使用するライブラリなどの情報がすべて記載されています。JSON (JavaScript Object Notation: 構造化データの記述形式) というフォーマットで書きます。

```

{
  "name": "book-management-app",
  "version": "1.0.0",
  "description": "書籍管理 Web アプリケーション",
  "main": "index.js",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "start": "node server.js",
    "test": "vitest"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "express": "^4.18.2"
  },
  "devDependencies": {
    "vite": "^5.0.0",
    "vitest": "^1.0.0",
    "typescript": "^5.3.0"
  }
}

```

// プロジェクト名。英小文字とハイフン
 // 自分のプロジェクトのバージョン（後
 // プロジェクトの簡単な説明文（人間向け）
 // このパッケージを他から読み込む時の
 // `npm run <キー>` で実行できるシ
 // `npm run dev` → vite（開発サー
 // `npm run build` → 本番用にビル
 // `npm run start` → node で ser
 // `npm run test` → vitest でテス
 // 本番でも必要なライブラリ。アプリ実
 // React本体
 // ReactをDOM（ブラウザ）に描画す
 // Webサーバーフレームワーク
 // 開発時だけ必要。本番デプロイには含
 // ビルドツール
 // テストツール
 // TypeScriptコンパイラ

各フィールドの意味:

フィールド	説明
<code>name</code>	プロジェクト名（小文字、ハイフン区切り）
<code>version</code>	プロジェクトのバージョン（セマンティックバージョンング）
<code>description</code>	プロジェクトの説明文
<code>scripts</code>	<code>npm run <名前></code> で実行できるコマンドの定義
<code>dependencies</code>	本番環境でも必要なライブラリ
<code>devDependencies</code>	開発時のみ必要なライブラリ（テストツール、ビルドツール等）

セマンティックバージョンング（semantic versioning：意味のあるバージョン番号付け、略してSemVer）：**メジャー**・**マイナー**・**パッチ** の3桁形式。たとえば **18.2.3** はメジャー18・マイナー2・パッチ3。- **メジャー**: 互換性が壊れる大きな変更があったとき増える - **マイナー**: 後方互換のある機能追加で増える - **パッチ**: バグ修正のみで増える

バージョン指定の記号

記号	意味	例
<code>^</code>	メジャーバージョン固定（マイナー・パッチは最新）	<code>^18.2.0</code> → 18.x.x の最新
<code>~</code>	メジャー・マイナー固定（パッチのみ最新）	<code>~18.2.0</code> → 18.2.x の最新
なし	完全固定	<code>18.2.0</code> → 18.2.0 のみ
<code>*</code>	任意のバージョン（非推奨）	<code>*</code> → 何でもOK

2.4 node_modules とは

`node_modules` フォルダは、`npm install` を実行したときにダウンロードされるライブラリの実体が格納される場所です。

```
my-project/
├─ node_modules/      ← ライブラリの実体（数千ファイル）
│   ├─ react/         ← React本体のフォルダ
│   ├─ react-dom/     ← ReactのDOM連携部分
│   ├─ express/       ← Express本体
│   └─ ... 数百のパッケージ（依存ライブラリの依存ライブラリも含まれる）
├─ package.json       ← 依存関係の定義（自分が書くファイル）
├─ package-lock.json  ← バージョンの完全固定（npmが自動生成）
└─ src/              ← あなたのコードを置く場所
```

⚠ **注意:** `node_modules` フォルダは非常に大きくなります（数百MB になることもあります）。Git にはコミットしないでください。後の章で設定する `.gitignore` ファイルで除外します。

💡 **ヒント:** `node_modules` を削除してしまっても、`npm install` を実行すれば `package.json` と `package-lock.json` の情報をもとにすべて復元できます。

2.5 基本的な npm コマンド

```
# プロジェクトの初期化 (カレントフォルダに package.json を新規作成)
# -y は --yes の短縮形。質問にすべてデフォルト値で「はい」と答えるフラグ
# これがないと「name は?」「version は?」と対話的に聞かれる
npm init -y

# 指定パッケージを dependencies としてインストール
# install を i と短く書いてもOK (npm i react)
# 自動で package.json と package-lock.json に追記される
npm install react

# 開発時のみ必要なパッケージを devDependencies としてインストール
# --save-dev は -D とも書ける。「dev に保存」の意味
# テストツールやビルドツールはこちらに入れるのが慣習
npm install --save-dev vitest

# 引数なし: package.json に書かれた全パッケージをまとめてインストール
# 別の人のプロジェクトを git clone した直後に最初にやる操作
npm install

# 指定パッケージを削除 (package.json から消える)
# uninstall は remove や rm でも同じ動作
npm uninstall react

# scripts セクションに定義したコマンドを実行
# 例: package.json の "dev": "vite" を実行する
npm run dev

# 直接インストール済みのトップレベルパッケージ一覧を表示
# --depth=0 は「依存の深さ0階層まで」つまり自分が直接入れたものだけ表示
# これを付けないと、依存の依存も全部出てきて画面が埋まる
npm list --depth=0

# 古くなったパッケージを一覧表示。Current / Wanted / Latest 列で確認可能
# outdated = 「時代遅れの」「古い」という意味
npm outdated
```


2.6 パッケージ管理のフロー



3. コードエディタ (VS Code)

3.1 VS Code とは

Visual Studio Code (VS Code) は、Microsoft が開発した **無料** のコードエディタ（プログラムを書くための専用テキストエディタ）です。軽量でありながら強力な機能を持ち、豊富な拡張機能によってあらゆるプログラミング言語に対応できます。Web 開発において最も人気のあるエディタの一つです。

3.2 インストール手順

Windows

1. [VS Code 公式サイト](#) にアクセス
2. 「Download for Windows」をクリック
3. ダウンロードした **.exe** ファイルを実行
4. インストールオプション: - 「PATHへの追加」にチェックを入れる（重要） - 「エクスプローラーのファイルコンテキストメニューに "Code で開く" を追加」にチェックを入れる（便利：ファイルを右クリックして直接VS Codeで開ける） - 「エクスプローラーのディレクトリコンテキストメニューに "Code で開く" を追加」にチェックを入れる（便利：フォルダを右クリックして開ける）
5. 「インストール」をクリック

 **注意:** 「PATHへの追加」にチェックを入れ忘れると、ターミナルから **code** コマンドで VS Code を起動できません。忘れた場合は再インストールしてください。

Mac

1. [VS Code 公式サイト](#) にアクセス
2. 「Download for Mac」をクリック
3. ダウンロードした **.zip** ファイルを展開（ダブルクリックで自動展開）
4. **Visual Studio Code.app** を「アプリケーション」フォルダにドラッグ
5. **code** コマンドを使えるようにする: - VS Code を起動 - **Cmd + Shift + P** でコマンドパレット（VS Code内のコマンド検索窓）を開く - 「Shell Command: Install 'code' command in PATH」を選択

Linux (Ubuntu/Debian)

```
# Microsoft の GPG キー（電子署名の鍵）とリポジトリ（配布元）を追加する手順

# 1. Microsoftの公開鍵をネット経由で取得 → バイナリ形式に変換 → ファイル保存
# wget -qO- : URLからダウンロードして標準出力に流す
# -q : quiet (進捗バーを表示しない)
# -O- : 出力先を標準出力に指定 (ハイフンが標準出力の意味)
# gpg --dearmor : テキスト形式の鍵をバイナリ形式に変換
# > packages.microsoft.gpg : 変換結果をファイルに保存 (上書きリダイレクト)
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor > packages.mic

# 2. 鍵ファイルを所定の場所へインストール (権限付き)
# install : ファイルを所定権限でコピーする専用コマンド
# -D : 必要なら親ディレクトリも作成
# -o root : 所有者を root に
# -g root : グループを root に
# -m 644 : 権限を 644 (所有者=読み書き、他=読みのみ) に
sudo install -D -o root -g root -m 644 packages.microsoft.gpg /etc/apt/keyrings/packages.mic

# 3. リポジトリ情報を apt のソースリストに追加
# echo "...": 文字列を出力
# | sudo tee <ファイル> : 管理者権限で出力をファイルに書き込む
# (sudo は echo に効かないのでこの書き方が必要)
# > /dev/null : tee の標準出力を捨てる (画面に二重表示しないため)
echo "deb [arch=amd64,arm64,armhf signed-by=/etc/apt/keyrings/packages.microsoft.gpg] http

# 4. 一時ファイルを削除
# rm : ファイル削除
# -f : 存在しなくてもエラーにしない (force)
rm -f packages.microsoft.gpg

# 5. パッケージ一覧を更新してインストール
sudo apt update
sudo apt install code -y

# 6. インストール確認
# --version : バージョン情報を表示するロングフラグ
code --version
```

3.3 推奨拡張機能

VS Code のサイドバーから拡張機能アイコン（四角が4つのアイコン）をクリックし、以下の拡張機能を検索してインストールしてください。

必須の拡張機能

拡張機能名	ID	用途	重要度
Japanese Language Pack	<code>MS-CEINTL.vscode-language-pack-ja</code>	VS Code の日本語化	必須
ESLint	<code>dbaeumer.vscode-eslint</code>	JavaScript/TypeScript のコード品質チェック	必須
Prettier - Code formatter	<code>esbenp.prettier-vscode</code>	コードの自動フォーマット	必須
ES7+ React/Redux/React-Native snippets	<code>dsznajder.es7-react-js-snippets</code>	React のコードスニペット	必須

強く推奨する拡張機能

拡張機能名	ID	用途	重要度
TypeScript Importer	<code>pmneo.tsimporter</code>	import 文の自動補完	推奨
Auto Rename Tag	<code>formulahendry.auto-rename-tag</code>	HTML/JSX タグの自動リネーム	推奨
Path Intellisense	<code>christian-kohler.path-intellisense</code>	ファイルパスの自動補完	推奨
GitLens	<code>eamodio.gitlens</code>	Git の履歴・変更を可視化	推奨
Error Lens	<code>usernamehw.errorlens</code>	エラーをエディタ上にインライン表示	推奨
Thunder Client	<code>rangav.vscode-thunder-client</code>	API テスト（Postman の代替）	推奨

あると便利な拡張機能

拡張機能名	ID	用途	重要度
Material Icon Theme	<code>PKief.material-icon-theme</code>	ファイルアイコンの見た目を改善	任意
indent-rainbow	<code>oderwat.indent-rainbow</code>	インデントを色分け表示	任意
Bracket Pair Color DLW	<code>BracketPairColorDLW.bracket-pair-color-dlw</code>	対応する括弧を色分け	任意
Code Spell Checker	<code>streetsidesoftware.code-spell-checker</code>	スペルチェック	任意

💡 ヒント: コマンドラインから一括インストールすることもできます: ``bash

code --install-extension : 指定IDの拡張機能をインストール

GUIで一つずつ検索する手間が省ける

```
code --install-extension MS-CEINTL.vscode-language-pack-ja # 日本語化
code --install-extension dbaeumer.vscode-eslint # コード品質チェック
code --install-extension esbenp.prettier-vscode # 自動フォーマッター
code --install-extension dsznajder.es7-react-js-snippets # React用スニペット
code --install-extension eamodio.gitlens # Git可視化
code --install-extension usernamehw.errorlens # エラーをインライン表示 ``
```

3.4 基本的な使い方

キーボードショートカット (Windows / Mac)

操作	Windows	Mac
コマンドパレット	Ctrl + Shift + P	Cmd + Shift + P
ファイル検索	Ctrl + P	Cmd + P
全文検索	Ctrl + Shift + F	Cmd + Shift + F
ターミナルの表示/非表示	Ctrl + `	Cmd + `
サイドバーの表示/非表示	Ctrl + B	Cmd + B
行の複製	Shift + Alt + ↓	Shift + Option + ↓
行の移動	Alt + ↑/↓	Option + ↑/↓
複数カーソル	Ctrl + Alt + ↑/↓	Cmd + Option + ↑/↓
名前の一括変更	F2	F2
定義へ移動	F12	F12
保存	Ctrl + S	Cmd + S
全保存	Ctrl + K, S	Cmd + Option + S

フォルダを開いて作業を始める

```
# 指定パスのフォルダをVS Codeで開く
# code は VS Code を起動するCLIコマンド（インストール時に PATH を通した）
# /path/to/my-project は開きたいフォルダの絶対パス
code /path/to/my-project

# カレントフォルダ（いま自分がいるフォルダ）をVS Codeで開く
# . は「現在のディレクトリ」を表す特殊な記号
code .
```

推奨する VS Code 設定

VS Code で **Ctrl + Shift + P** (Mac: **Cmd + Shift + P**) を押して「Preferences: Open Settings (JSON)」を選択し、以下の設定を追加します。

```
{
  "editor.formatOnSave": true, // ファイル保存時に自動でフォーマット
  "editor.defaultFormatter": "esbenp.prettier-vscode", // デフォルトフォーマッターを Prettier
  "editor.tabSize": 2, // Tab1回あたりのスペース数を2に
  "editor.wordWrap": "on", // 長い行を画面幅で折り返して表示
  "editor.minimap.enabled": false, // 右側のミニマップ（縮小プレビュー）を非表示
  "editor.bracketPairColorization.enabled": true, // 対応する括弧をペアごとに色分け
  "editor.guides.bracketPairs": "active", // カーソル位置の括弧ガイドラインを表示
  "files.autoSave": "onFocusChange", // 別のファイルに切り替えた瞬間に自動保存
  "terminal.integrated.defaultProfile.windows": "Git Bash", // Windowsの統合ターミナルをGit Bashに変更
  "emmet.includeLanguages": { // Emmet (HTML 省略記法) を有効化する
    "javascript": "javascriptreact" // .js ファイルでも JSX 用の Emmet 記法を使用
  }
}
```

設定	効果
<code>formatOnSave</code>	ファイル保存時に自動フォーマット
<code>defaultFormatter</code>	Prettier をデフォルトフォーマッターに設定
<code>tabSize: 2</code>	インデント幅を2スペースに設定
<code>wordWrap</code>	長い行を折り返して表示
<code>autoSave</code>	フォーカスが外れたら自動保存
<code>defaultProfile.windows</code>	Windows のデフォルトターミナルを Git Bash に変更

4. Git の基礎

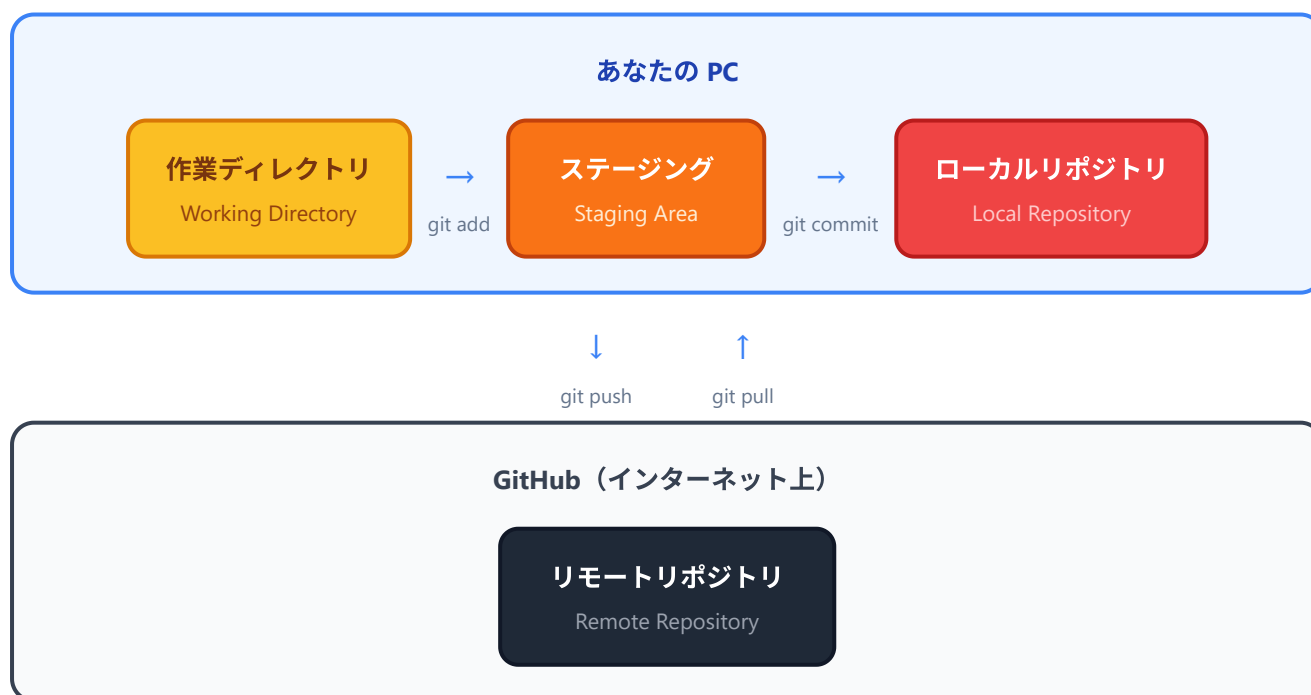
4.1 Git とは

Git は **バージョン管理システム**（Version Control System：VCS）です。ファイルの変更履歴を記録し、過去の状態に戻したり、複数人で同時に開発したりすることを可能にします。

Git用語の予習: - **リポジトリ**（repository：略してリポ／レポ）：プロジェクトの履歴を全部入れておく「箱」。1プロジェクトに1つ作るのが基本。 - **コミット**（commit）：「ここまでの変更を1つの履歴として保存する」操作・記録。スナップショット（瞬間記録）と考えるとイメージしやすい。 - **ブランチ**（branch：枝）：履歴を枝分かれさせる仕組み。新機能を試すために本流から分岐 → うまくいったら本流に合流（マージ）する。 - **リモート**（remote）：自分のPCの外（GitHubなど）にあるリポジトリ。それに対して自分のPC内のものは「ローカル」と呼ぶ。

なぜ Git が必要なのか

状況	Git なし	Git あり
コードを壊してしまった	元に戻せない	<code>git checkout</code> で復元可能
誰がいつ変更したか知りたい	わからない	<code>git log</code> で確認可能
新機能を試したい	メインのコードを壊すリスク	ブランチで安全に実験可能
複数人で開発	ファイルの上書き事故	マージ機能で安全に統合
コードを公開・共有したい	USBやメールで送る	GitHub にプッシュ



4.2 Git のインストール

Windows

1. [Git for Windows](#) にアクセス
2. インストーラーをダウンロードして実行
3. インストールオプション（以下のデフォルト推奨設定を確認）： - **Default editor**: 「Use Visual Studio Code as Git's default editor」を選択（コミットメッセージなどを書くときに開かれるエディタの指定） - **Adjusting your PATH:** 「Git from the command line and also from 3rd-party software」を選択（コマンドプロンプトからもGitを使えるようにする） - **Line ending conversions**: 「Checkout Windows-style, commit Unix-style line endings」を選択（改行コードの自動変換設定。詳細は後述） - その他はデフォルトのまま「Next」を押してインストール

💡 **ヒント**: Git for Windows をインストールすると **Git Bash** というターミナルも一緒にインストールされます。Linux/Mac と同じコマンドが使えるので非常に便利です。

Mac

```
# 方法1: Xcode Command Line Tools (Apple純正の開発ツール一式) を入れる
# これに Git が同梱されている。初回はインストールダイアログが出る
xcode-select --install

# 方法2: Homebrew (Mac用パッケージマネージャー) で入れる（推奨：最新版が使える）
# brew install <パッケージ名> でインストール
brew install git
```

Linux (Ubuntu/Debian)

```
# パッケージ一覧を最新化
sudo apt update
# git をインストール。-y で確認プロンプトを自動承認
sudo apt install git -y
```

バージョン確認

```
# Git のバージョンを表示
# --version はロングフラグ。多くのコマンドで共通
git --version
# 出力例: git version 2.43.0
```

4.3 Git の初期設定

インストール後、ユーザー名とメールアドレスを設定します。これはコミット（変更の記録）に「誰が変更したか」として記録される情報です。


```
# git config: Git の設定を読み書きするコマンド
# --global: PC全体（全プロジェクト共通）に保存。
#           付けないと現在のリポジトリだけの設定になる
# user.name: 設定キー（名前空間: 設定名）。コミット作者の名前を表す
# "...": 設定値。スペースを含むので引用符必須
git config --global user.name "あなたの名前"

# メールアドレスを設定（GitHub アカウントと同じものを推奨）
# user.email: コミット作者のメールアドレス
git config --global user.email "your-email@example.com"

# 新規リポジトリ作成時のデフォルトブランチ名を main に設定
# 古い慣習では master だったが、現在は main が標準
git config --global init.defaultBranch main

# 現在の全グローバル設定を確認
# --list : 設定一覧を表示するフラグ
git config --global --list
```

4.4 基本コマンド

リポジトリの作成と基本操作

Git の基本フローを「ファイル作成 → ステージ → コミット」の流れで体験します。各コマンドが何をしているか1行ずつ解説します。

「同じディレクトリにいる」って何?: ターミナルは「いま開いているフォルダ」が常に1つあって、それを **カレントディレクトリ** と呼びます。 **pwd** で確認できます。以下のコマンドは「 **my-project** フォルダの中にいる状態」で実行することを前提にしています。フォルダから出してしまうと、別のフォルダに対して操作してしまうので注意。

```

# -----
# (1) 新しい作業フォルダを作って、その中に移動する
# -----
# mkdir = "make directory" の略。フォルダを新規作成するコマンド。
# 引数 my-project は作成するフォルダ名
mkdir my-project

# cd = "change directory" の略。カレントディレクトリ（今いる場所）を移動。
# これ以降のコマンドはこの my-project フォルダ内で動く
cd my-project

# -----
# (2) Git リポジトリを初期化する
# -----
# git init は「このフォルダをGit管理下にする」コマンド。
# 隠しフォルダ .git/ が作られ、ここに変更履歴が全部保存される。
# 一度実行すれば良い。再実行しても害はないが普通は不要
git init
# ▼ 期待する出力
# Initialized empty Git repository in /path/to/my-project/.git/

# -----
# (3) サンプルファイルを1個作る
# -----
# echo "... " は文字列をそのまま出力するコマンド。
# > README.md は「出力先をファイルに切り替える」リダイレクト演算子。
# 結果として「README.md という名前のファイルに "# My Project" と書く」操作になる。
# > は上書き、>> は追記。ここでは新規作成なので > を使用
echo "# My Project" > README.md

# -----
# (4) 現在のリポジトリの状態を確認する
# -----
# git status は「いま何が変わってる？」を教えてくれる。最頻出コマンド。
# 引数なしで何度実行してもファイルを変えない（安全に確認用に使える）
git status
# ▼ 期待する出力（要約）
# On branch main
# No commits yet
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#       README.md
# nothing added to commit but untracked files present (use "git add" to track)
# ▼ 「Untracked」= まだGitに認識されていない新規ファイル、の意味。

```

```

# -----
# (5) ファイルを「ステージングエリア」に追加
# -----
# Gitは2段階で変更を記録する。
# ①ステージング (git add) ← どの変更を記録するかを選ぶ
# ②コミット (git commit) ← 選んだ変更をスナップショットとして記録
# こうすることで「複数の変更のうち一部だけまとめてコミット」ができる。
git add README.md          # 引数 = 特定ファイル名。そのファイルだけ追加
git add .                  # 引数 . は「カレントフォルダ以下の全変更」。よく使う

# -----
# (6) コミットする = スナップショットを履歴に記録
# -----
# -m "... " はコミットメッセージを指定するフラグ。message の m。
# なぜこの変更をしたかを短文で残す。必ず引用符で囲む。日本語OK。
# -m を付けるとデフォルトエディタ (vim/VS Code) が開いてメッセージ入力を求められる
git commit -m "最初のコミット: READMEを追加"
# ▼ 期待する出力 (要約)
# [main (root-commit) 1a2b3c4] 最初のコミット: READMEを追加
# 1 file changed, 1 insertion(+)
# create mode 100644 README.md

# -----
# (7) コミット履歴を見る
# -----
# git log だけだと1コミットあたり数行使うので画面が長くなる。
# --oneline で1行に圧縮表示できる。
git log                    # 通常表示: 1コミットあたり 作者/日時/メッセージ で4-5行
git log --oneline          # 1行表示: ハッシュとメッセージのみ。一覧性◎
# ▼ 出力例 (git log --oneline)
# 1a2b3c4 (HEAD -> main) 最初のコミット: READMEを追加

```

リモートリポジトリとの連携

```
# -----
# (1) リモートリポジトリ「origin」を登録
# -----
# git remote add <名前> <URL>: 別の場所にあるGitリポジトリに別名 (origin) を付ける。
# origin は慣習的に「メインのリモート」を指す名前。GitHubで作ったリポジトリのURLを使う。
# 一度登録すれば次回以降は名前 origin だけで参照できる
git remote add origin https://github.com/ユーザー名/リポジトリ名.git

# -----
# (2) ローカルのコミットをGitHubに送る（プッシュ）
# -----
# git push <リモート名> <ブランチ名>
# -u は --set-upstream の短縮形。「次回からは git push だけで origin main に送る」設定。
# 初回プッシュ時にのみ付ければOK
git push -u origin main
# ▼ 出力例
# Enumerating objects: 3, done.
# Counting objects: 100% (3/3), done.
# Writing objects: 100% (3/3), 234 bytes | 234.00 KiB/s, done.
# Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
# To https://github.com/yourname/my-project.git
# * [new branch]      main -> main
# branch 'main' set up to track 'origin/main'.

# -----
# (3) GitHubの最新コミットをローカルに取り込む（プル）
# -----
# 他の開発者やGitHub上で行われた変更を、自分のローカルに反映する。
# git pull = git fetch (取得) + git merge (統合) の2段階を一気にやるコマンド
git pull origin main
```

よく使うコマンドの一覧

コマンド	説明
<code>git init</code>	リポジトリを初期化
<code>git status</code>	変更状態を確認
<code>git add <ファイル></code>	ステージングエリアに追加
<code>git add .</code>	すべての変更をステージに追加
<code>git commit -m "メッセージ"</code>	変更をコミット
<code>git log --oneline</code>	コミット履歴を簡潔に表示
<code>git diff</code>	変更内容の差分を表示
<code>git branch</code>	ブランチ一覧を表示
<code>git checkout -b <名前></code>	新しいブランチを作成して切り替え
<code>git push</code>	リモートにプッシュ
<code>git pull</code>	リモートからプル
<code>git clone <URL></code>	リポジトリをコピー

4.5 GitHub アカウントの作成

1. [GitHub](#) にアクセス
2. 「Sign up」をクリック
3. メールアドレス、パスワード、ユーザー名を入力
4. メール認証を完了
5. 無料プラン（Free）を選択

SSH キーの設定（推奨）

パスワード入力なしで GitHub にプッシュできるように SSH キー（暗号化された電子鍵）を設定します。

SSHキーとは:「公開鍵」と「秘密鍵」のペア。公開鍵をGitHubに登録し、秘密鍵を自分のPCに置いておくと、毎回パスワードを打たずに安全に通信できる仕組み。

```
# SSH キーペア（公開鍵と秘密鍵）を生成する
# ssh-keygen: SSH鍵を作るコマンド
# -t ed25519 : 鍵の種類 (type)。ed25519 は最新の暗号方式で推奨
# -C "..." : コメント (Comment)。鍵の識別ラベル。メアドにするのが慣習
ssh-keygen -t ed25519 -C "your-email@example.com"
# 実行中に3回プロンプトが出る:
# 1) 保存先 → デフォルト ~/.ssh/id_ed25519 のままEnter
# 2) パスフレーズ → 空のままEnter（または任意のパス）
# 3) パスフレーズ確認 → 同じくEnter

# 公開鍵（公開しても安全な方の鍵）の中身を画面に表示
# cat: ファイル内容を表示するコマンド
# ~/.ssh/id_ed25519.pub : .pub が公開鍵の慣習的な拡張子
cat ~/.ssh/id_ed25519.pub
# 表示された ssh-ed25519 で始まる1行をすべてコピー（メアド部分含む）
```

GitHub での設定:

1. GitHub にログイン
2. 右上のアイコン → 「Settings」
3. 左メニュー「SSH and GPG keys」
4. 「New SSH key」をクリック
5. Title: 任意の名前（例: "My PC"）
6. Key: コピーした公開鍵を貼り付け
7. 「Add SSH key」をクリック

```
# GitHubに接続できるかテスト
# ssh -T <ユーザー@ホスト> : テスト接続。-T はリモートシェル割り当て無効
# git@github.com : GitHubのSSH接続先（ユーザー名は固定で git）
ssh -T git@github.com
# 出力: Hi ユーザー名! You've successfully authenticated...
# 初回は「本当に接続しますか？」と聞かれるので yes と入力
```

4.6 Git トラブルシューティング

push 時に認証エラー

```
remote: Support for password authentication was removed
```

原因: GitHub はパスワード認証を廃止しました（2021年8月以降）。

解決方法: SSH キーを設定するか、Personal Access Token（PAT：個人用アクセストークン）を使用します。上記の SSH キーの設定を参照してください。

改行コードの警告 (Windows)

```
warning: LF will be replaced by CRLF
```

改行コードの違い: WindowsはCR+LF (`\r\n` の2文字)、Mac/LinuxはLFのみ (`\n` の1文字) で改行を表します。チームで開発するところで衝突するため、Gitが自動変換します。

解決方法:

```
# Windows用設定: チェックアウト時に LF→CRLF へ自動変換し、コミット時にCRLF→LFへ戻す
# core.autocrlf : 改行コード自動変換の設定キー
#   true : Windowsスタイル変換 (推奨)
#   input : Mac/Linux用設定
#   false : 変換しない
git config --global core.autocrlf true
```

コミットメッセージのエディタが vim で困る

```
# Gitがエディタを開く場面 (git commit を -m なしで実行など) で使うエディタを変更
# core.editor : エディタを指定する設定キー
# "code --wait" : VS Code を起動し、--wait でファイルが閉じられるまで待つ
git config --global core.editor "code --wait"
```

5. ターミナル/コマンドラインの基礎

5.1 ターミナルとは

ターミナル (コマンドライン、シェル、CLI と呼ばれます) は、テキストでコンピューターに指示を出すためのツールです。GUI (マウスでクリックする画面) では操作が複雑になるような作業も、コマンド一つで実行できます。

Web 開発ではターミナルを頻繁に使うため、基本的なコマンドを覚えておくことが重要です。

ターミナルの画面はこんな見た目

初めてターミナルを開くと、こんな画面が出てきます (黒や白の背景に、緑や白の文字)。

```
yuya@MyPC MINGW64 ~/Desktop
$ _
```

各部分の意味は次のとおりです。

部分	例	意味
ユーザー名	yuya	ログイン中のユーザーの名前
PC名	MyPC	あなたのコンピュータの名前
シェル種別	MINGW64	Git Bash の場合に表示される識別子。PowerShellなら別の表示
カレントディレクトリ	~/Desktop	「いま自分がいる場所」。~ はホームフォルダ
プロンプト	\$ または >	「ここからコマンドを打ってね」という合図
カーソル	_ (点滅)	入力位置を示す光る四角

「カレントディレクトリ」って何?: いまターミナルが「自分の現在地」として認識しているフォルダのことです。pwd コマンドで確認でき、cd コマンドで移動できます。Windowsのエクスプローラーで「現在開いているフォルダ」と同じ概念です。

コマンドを「実行する」とは？

たとえば pwd というコマンドを実行する手順は以下のとおりです。

1. プロンプト \$ の右側にカーソルが点滅していることを確認
2. キーボードで pwd と打つ
3. Enterキー を押す
4. すぐ下の行に結果（出力）が表示される
5. その下に新しいプロンプトが現れて、次のコマンドを待つ

▼ 実行例:

```
$ pwd
/c/Users/yuya/Desktop
$ _
```

ここで pwd の右下に表示された /c/Users/yuya/Desktop が、コマンドの実行結果です。これは「いまDesktopフォルダにいるよ」という意味です。

行頭の \$ や > はコピーしない

このチュートリアル中で出てくるサンプルコマンドの中には、行頭に \$ が付いているものがあります。

```
$ node -v
```

この \$ は「これはターミナルに打つコマンドですよ」という印で、実際には打ちません。打つのは \$ より右の node -v だけです。同じく PowerShell のサンプルで PS C:\> のような表示があったら、それも飾りです。

代表的なプロンプト記号: - **\$** : 一般ユーザー (Bash, Git Bash, zsh など) - **#** : 管理者 (root) ユーザー - **>**
: Windows コマンドプロンプト - **PS C:\>** : PowerShell (**PS** の後にカレントパスが続く)
本書では混乱を避けるため、コマンドだけを書いてプロンプト記号は省略するスタイルを基本にしています。

5.2 Windows のターミナル選択肢

Windows には複数のターミナルがあります。

ターミナル	特徴	推奨度
Git Bash	Linux/Mac と同じコマンドが使える。Git に同梱	★★★★★
Windows Terminal	Microsoft 公式。複数のシェルをタブで使い分け可能	★★★★☆
PowerShell	Windows 標準。独自のコマンド体系	★★★☆☆
コマンドプロンプト (cmd)	旧来の Windows ターミナル。機能が限定的	★★☆☆☆

 **ヒント:** 本チュートリアルでは **Git Bash** の使用を推奨します。オンラインの記事やドキュメントの多くは Linux/Mac のコマンドで書かれており、Git Bash ならそれらをそのまま実行できます。

Git Bash の起動方法

- スタートメニューで「Git Bash」と検索して起動
- 任意のフォルダで右クリック → 「Git Bash Here」
- VS Code のターミナルを Git Bash に設定 (前述の設定で対応済み)

PowerShell と Git Bash のコマンド比較

操作	Git Bash / Mac / Linux	PowerShell
現在のフォルダ表示	<code>pwd</code>	<code>pwd</code> または <code>Get-Location</code>
ファイル一覧	<code>ls</code>	<code>ls</code> または <code>Get-ChildItem</code>
フォルダ移動	<code>cd folder</code>	<code>cd folder</code>
フォルダ作成	<code>mkdir folder</code>	<code>mkdir folder</code>
ファイル削除	<code>rm file.txt</code>	<code>Remove-Item file.txt</code>
フォルダ削除	<code>rm -rf folder</code>	<code>Remove-Item -Recurse folder</code>
ファイル内容表示	<code>cat file.txt</code>	<code>Get-Content file.txt</code>
テキスト検索	<code>grep "text" file</code>	<code>Select-String "text" file</code>
環境変数表示	<code>echo \$PATH</code>	<code>\$env:PATH</code>
画面クリア	<code>clear</code>	<code>cls</code> または <code>Clear-Host</code>

5.3 基本コマンド一覧

以下は、開発で頻繁に使うコマンドです（Git Bash / Mac / Linux 共通）。

ファイル・フォルダ操作

```
# 現在のフォルダ（ディレクトリ）のフルパスを表示
# pwd = "print working directory" の略
pwd
# 出力例: /c/Users/yuya/Desktop/education

# カレントディレクトリ内のファイル・フォルダー一覧を表示
# ls = "list" の略
ls
# 出力例: Documents Downloads Desktop

# 隠しファイル（. で始まるファイル）も含めた詳細一覧
# ls にフラグを2つ付ける書き方
# -l : long format（詳細表示）。権限・所有者・サイズ・日時など
# -a : all（隠しファイルも含む）
# -la と並べて書いてもOK
ls -la
# 出力例:
# drwxr-xr-x  5 yuya  staff  160  1  1 12:00 .
# drwxr-xr-x  3 yuya  staff   96  1  1 12:00 ..
# -rw-r--r--  1 yuya  staff   50  1  1 12:00 .gitignore
# -rw-r--r--  1 yuya  staff  200  1  1 12:00 package.json

# フォルダの移動
cd Documents      # サブフォルダ Documents に入る
cd ..             # .. は「一つ上」の意味。親フォルダへ移動
cd ~              # ~ はホームフォルダ。引数なしの cd でも同じ
cd /c/Users/yuya  # 絶対パスで移動（Git BashでのWindowsパス表記）

# フォルダの作成
mkdir my-project      # 単一フォルダ作成
# -p は parents の略。途中のフォルダがなくても順に作成
# 普通の mkdir は親が無いとエラーになるが、-p は中間フォルダも自動生成
mkdir -p src/components/ui      # src/components/ui を一気に作成

# ファイルの作成
# touch : ファイルの「最終更新日時」を現在に変更するコマンド
# ファイルが存在しなければ「空ファイル」として作成される副作用がある
touch index.html
# echo "... " > file : echoの出力をファイルに上書き保存
echo "Hello" > hello.txt

# ファイルのコピー
# cp = "copy" の略
cp file.txt file-backup.txt      # 第1引数を第2引数の名前でコピー
# -r は recursive（再帰的）の略。フォルダごとコピーする場合に必須
cp -r src/ src-backup/

# ファイルの移動・リネーム
```

```
# mv = "move" の略。リネームも実は「同じフォルダ内への移動」扱い
mv old-name.txt new-name.txt      # ファイル名変更
mv file.txt Documents/            # ファイルをフォルダに移動

# ファイルの削除
# rm = "remove" の略。ゴミ箱を経由せず即削除
rm file.txt

# -r : recursive (フォルダ内も再帰的に削除)
# -f : force (確認なしで強制実行、存在しないファイルでもエラーにしない)
# 合わせて -rf で「フォルダごと黙って削除」になる。便利だが危険
rm -rf node_modules/
```

⚠ **注意:** `rm -rf` は確認なしに完全削除されます。特に `rm -rf /` や `rm -rf ~` のような広範囲の削除は絶対に実行しないでください。取り返しがつきません。

ファイル内容の表示

```
# ファイル全体を画面に出力
# cat = "concatenate" の略 (本来は複数ファイル連結用コマンド)
cat package.json

# 長いファイルをページ単位でスクロール表示
# less は「ページャー」と呼ばれる対話的閲覧プログラム
# 操作: スペース=次ページ / b=前ページ / / で検索 / q で終了
less README.md
# (q キーで終了)

# ファイル先頭から指定行数だけ表示
# head = 「頭」の意味
# -n 10 : 表示する行数を10に指定 (n は number の n)
head -n 10 server.js

# ファイル末尾から指定行数だけ表示
# tail = 「尻尾」の意味。ログの最新行を見るのによく使う
tail -n 10 server.log
```

その他の便利なコマンド

```
# 画面をクリアして上のほうの出力履歴を見えないところにする
clear

# 過去に打ったコマンドの履歴を一覧表示
# 上矢印キー ↑ でも1つずつ呼び出せる
history

# 指定コマンドの実行ファイルがどこにあるか調べる
# which = 「どこ」の意味
which node
# 出力例: /home/yuya/.nvm/versions/node/v20.11.0/bin/node

# ファイル・フォルダを検索
# find <検索場所> <条件>
# . : カレントフォルダから検索開始
# -name "*.js" : ファイル名が *.js (任意の名前.js) にマッチ
find . -name "*.js"

# ファイル内のテキストを検索
# grep <パターン> <ファイル>
grep "import" src/App.tsx
# -r : recursive。フォルダを指定するとその中を再帰的に検索
grep -r "TODO" src/
```

5.4 パス (Path) の基本

パスとは: ファイルやフォルダの場所を表す住所のような文字列。書き方が2種類あります。

```
# 絶対パス (absolute path) : ルート (/) から始まる完全な住所
# どこから打っても同じ場所を指す
/c/Users/yuya/Desktop/education/my-project/src/App.tsx

# 相対パス (relative path) : 現在のフォルダを基準にした書き方
# どこにいるかで指す先が変わる
./src/App.tsx      # ./ は「ここ」。カレントフォルダ内の src/App.tsx
../other-project/  # ../ は「一つ上」。親フォルダの中の other-project
../../            # ../../ で「二つ上」。連結で何段でも遡れる

# 特殊なパス記号
~                  # チルダ。あなたのホームフォルダ (例: /c/Users/yuya)
.                  # ピリオド単独。現在のフォルダ
..                 # ピリオド2つ。一つ上のフォルダ
```

5.5 ターミナルのトラブルシューティング

コマンドが見つからない

```
bash: some-command: command not found
```

確認手順:

```
# コマンドの実体ファイルがどこにあるか確認
# 何も出なければ未インストール、または PATH 未登録
which some-command

# 環境変数 PATH の中身を表示
# echo : 文字列出力
# $PATH : 変数 PATH の値を参照 ($ が変数参照の記号)
echo $PATH

# PATH にコマンドの場所が含まれていない場合
# → 該当ツールを再インストールするか、PATH を設定する
```

日本語が文字化けする (Windows)

```
# Git Bash の場合、以下を ~/.bashrc に追加
# export : 環境変数を「子プロセスにも引き継がれる形」で設定するキーワード
# LANG : ロケール (言語と文字コード) の設定キー
# ja_JP.UTF-8 : 日本語(日本)、UTF-8文字コードという意味
export LANG=ja_JP.UTF-8
```

Tab 補完を活用する

```
# ファイル名やフォルダ名を途中まで入力して Tab キーを押すと残りが自動入力される
# タイポ防止と入力短縮の両方に役立つ最重要テクニック
cd Doc<Tab>
# → cd Documents/ に自動補完される

# 候補が複数ある場合は Tab を2回押すと一覧が表示される
cd D<Tab><Tab>
# → Desktop/ Documents/ Downloads/
```

6. 動作確認

すべてのツールが正しくインストールされたか、以下のチェックリストで確認しましょう。

6.1 確認コマンドの実行

ターミナル（Git Bash 推奨）を開いて、以下のコマンドを順番に実行してください。

```
# echo : 引数の文字列を画面に出力するコマンド。シェルスクリプトの「表示」担当
# "... " 内は1行の文字列として扱われる
echo "=== 開発環境チェック ==="
# 引数なしの echo は空行を出力。見やすさのため挿入
echo ""

echo "1. Node.js:"
node -v                                # Node.js のバージョン表示
echo ""

echo "2. npm:"
npm -v                                 # npm のバージョン表示
echo ""

echo "3. Git:"
git --version                          # Git のバージョン表示
echo ""

echo "4. VS Code:"
code --version                         # VS Code のバージョン表示 (3行出力)
echo ""

echo "=== チェック完了 ==="
```

▼ 期待する実行結果（バージョンの数字は時期によって変わります）：

```
=== 開発環境チェック ===

1. Node.js:
v20.11.0

2. npm:
10.2.4

3. Git:
git version 2.43.0.windows.1

4. VS Code:
1.85.1
929bacba01ef7d04dac26da25e5dafdc4d039aaf
x64

=== チェック完了 ===
```

▼ 結果の読み方:

- `node -v` の `v` は version の v。続く数字が Node.js のメジャー・マイナー・パッチ バージョン
- `npm -v` は数字のみ。10.x なら本書の手順で問題なく動作する
- `git --version` の末尾 `.windows.1` は Windows用ビルドの印
- `code --version` は3行出力される（バージョン番号、コミットハッシュ、CPU種別）

▼ もし `command not found` などのエラーが出たら:

- `node: command not found` → Node.jsが未インストール。1.2 章に戻ってインストール
- `git: command not found` → Gitが未インストール。4.2 章に戻ってインストール
- `code: command not found` → VS Codeのインストールはできているが、`code` コマンドのパスが通っていない。VS Codeのコマンドパレット（`Ctrl+Shift+P`）で「Shell Command: Install 'code' command in PATH」を実行

6.2 チェックリスト

以下のすべてにチェックが入れば、環境構築は完了です。

- [] Node.js がインストールされている（`node -v` で `v18.0.0` 以上が表示される）
- [] npm がインストールされている（`npm -v` でバージョンが表示される）
- [] Git がインストールされている（`git --version` でバージョンが表示される）
- [] Git の初期設定 が完了している（`git config --global user.name` で名前が表示される）
- [] VS Code がインストールされている（`code --version` でバージョンが表示される）
- [] VS Code の日本語化 が完了している（メニューが日本語で表示される）
- [] VS Code の必須拡張機能がインストールされている（ESLint, Prettier, React snippets）
- [] GitHub アカウント を作成済み
- [] ターミナル で基本コマンドが実行できる（`pwd`, `ls`, `cd` など）

6.3 簡単なテスト

最後に、環境が正しく動作するかを実際に試してみましょう。


```

# 1. テスト用フォルダを作成して移動
# mkdir : フォルダ作成
# ~/Desktop : ホーム配下のDesktop。Mac/Linux/Git Bashで共通
mkdir ~/Desktop/env-test
cd ~/Desktop/env-test

# 2. Node.js プロジェクトを初期化
# npm init で package.json を作成
# -y : すべての質問にデフォルト値(yes)で答える
npm init -y

# 3. 簡単な JavaScript ファイルを作成
# cat > <ファイル> << 'EOF' ... EOF はヒアドキュメント構文
# cat : 引数なしだと標準入力をそのまま出力
# > file: 出力をファイルに保存
# << : ヒアドキュメント開始 (複数行入力を流し込む)
# 'EOF' : 終端マーカー。シングルクォートで囲むと $変数を展開しない (ここでは ${nodeVersion} などを
cat > hello.js << 'EOF'
const message = "環境構築が完了しました！";
const nodeVersion = process.version;
const platform = process.platform;

console.log("=====");
console.log(message);
console.log(`Node.js バージョン: ${nodeVersion}`);
console.log(`プラットフォーム: ${platform}`);
console.log("=====");
console.log("");
console.log("次の章では、プロジェクトの作成を始めます。");
console.log("お疲れ様でした！");
EOF

# 4. 作成した JavaScript ファイルを Node.js で実行
# node <ファイル名> : 指定ファイルを実行
node hello.js

```

以下のような出力が表示されれば成功です。

```

=====
環境構築が完了しました！
Node.js バージョン: v20.11.0
プラットフォーム: win32
=====

次の章では、プロジェクトの作成を始めます。
お疲れ様でした！

```

```
# 5. Git の動作確認
cd ~/Desktop/env-test      # 念のためテストフォルダに移動
git init                   # このフォルダを Git 管理下にする (.git/ ができる)
git add .                  # カレントフォルダ以下の全変更をステージに追加
git commit -m "環境テスト: 初回コミット" # 履歴にコミットを記録
git log --oneline          # コミット履歴を1行表示で確認

# 6. テストフォルダを削除 (任意)
# rm -rf : 中身ごとフォルダを強制削除。実行前に必ずパスを確認
cd ~/Desktop               # 削除対象から外に出る (中にいると削除に失敗する場合あり)
rm -rf env-test
```

6.4 問題が解決しない場合

上記の手順で問題が解決しない場合は、以下を試してください。

1. PC を再起動する — 環境変数の変更が反映されていない可能性があります
2. ターミナルを新しく開き直す — 設定の再読み込みが必要な場合があります
3. ツールを再インストールする — インストール時にオプションを間違えた可能性があります
4. エラーメッセージで検索する — エラーメッセージをそのまま Google で検索すると、解決策が見つかることが多いです

まとめ

この章では、以下のツールをインストールし、設定しました。

ツール	用途	確認コマンド
Node.js	JavaScript の実行環境	<code>node -v</code>
npm	パッケージマネージャー	<code>npm -v</code>
VS Code	コードエディタ	<code>code --version</code>
Git	バージョン管理	<code>git --version</code>
GitHub	コードの共有・ホスティング	ブラウザでログイン確認

次の章では、これらのツールを使って実際に書籍管理アプリのプロジェクトを作成していきます。

 **ヒント:** 環境構築は最初の一回だけです。一度セットアップが完了すれば、今後の章ではコードを書くことに集中できます。ここまでお疲れ様でした！

次の章へ: [第2章 TypeScript の基礎](#) →